Field Hours

CSCE 635 AI Robotics

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

How far each wheel rolled

You track a differential-drive robot by what each wheel traveled, not by a single speed. Let the right wheel roll 1.2 meters and the left roll 1.0 meter, with the wheels 0.5 meters apart. The body center moves the average, (1.2 plus 1.0) divided by 2, which is 1.1 meters. The heading changes by (1.2 minus 1.0) divided by 0.5, which is 0.4 radians. Apply heading first, then translation, and convert that body step into the world frame.

Wheel slip makes that 0.4 radian figure a prediction, not a measurement. Equal skids of 0.05 meters shift distance by 0.05 and leave heading error near 0. If they skid differently, heading absorbs (right skid minus left skid) divided by 0.5. You correct the prediction with a sensor later. For now, log both wheel distances and the pose you integrated. Do not hand that pose to a planner as if it were exact.

One body step from two wheel travels.

1. Average the wheel travels

2. Divide the difference by 0.5

3. Rotate, then translate

4. Write the pose in the world

1.1 meters and 0.4 radians

1. Right travel is 1.2 meters and left travel is 1.0 meter. The baseline is 0.5 meters.

2. Center travel is (1.2 plus 1.0) divided by 2, so 1.1 meters. Heading change is (1.2 minus 1.0) divided by 0.5, so 0.4 radians.

3. You apply the 0.4 radian turn first, then move 1.1 meters along the new heading. Equal skids of 0.05 meters would change the 1.1 and leave the 0.4 nearly alone.

Problem 1. Right wheel 1.2 meters, left wheel 1.0 meter, baseline 0.5 meters. What are the center travel and the heading change?

Answer. 1.1 meters and 0.4 radians. The center travel is the average of 1.2 and 1.0. The heading change is 0.2 divided by 0.5.

Problem 2. Both wheels skid an extra 0.05 meters in the same direction. Which of the two outputs moves, and by how much?

Answer. The center travel moves by 0.05 meters. The heading change stays near 0.4 radians, because equal skids cancel in the difference.

Lecture 2 · 90 minutes

A grid of occupied and free

You keep an occupancy grid, not just a pose. A cell of 0.2 meters turns a 10 meter side into 50 cells. Store log odds as an integer that starts at 0. Add 1 on a hit and subtract 1 on a miss. The updates plus 1, minus 1, minus 1 leave the cell at minus 1, which you call free. Your planner may cross a cell still at 0 only if you allow it.

A beam updates a whole ray, not one cell. Walk from the sensor out to the hit. Cells along the ray before the hit each get minus 1. The hit cell gets plus 1. A beam that travels 2 meters at 0.2 meters per cell crosses 10 cells: 9 free updates and 1 occupied update. You cap the ray at 4 meters, 20 cells, so a miss does not erase a wall.

One beam writes a ray of cells.

Walk out to the hit -> Subtract 1 before the hit -> Add 1 on the hit cell -> Stop by 4 meters

Ten cells along a 2 meter ray

1. The cell is 0.2 meters. A hit at 2 meters is 10 cells out.

2. You subtract 1 from the first 9 cells and add 1 to the 10th. A cell that started at 0 and was only the hit ends at plus 1, occupied.

3. A missed return must not walk past 4 meters. That cap is 20 cells. Without it you would mark free space through a wall you never saw.

Problem 1. Start at 0 and apply plus 1, minus 1, minus 1. What is the cell, and do you call it free?

Answer. Minus 1, and yes, you call a negative cell free. Zero would have stayed unknown.

Problem 2. A square room is 10 meters on a side and each cell is 0.2 meters. How many cells is that?

Answer. 2500. Each side is 50 cells, and 50 times 50 is 2500.

Lecture 3 · 90 minutes

Particles that vote for a pose

Odometry drifts, so you keep many pose guesses. With 100 particles you move each one by the wheel update plus noise, then weight it by the sensor match at that pose. Resampling draws a new set of 100, with replacement, using the weights as probabilities. A particle whose weight is 0 is never drawn. If one particle holds half the weight, you expect about 50 copies of it.

The sensor weight is a number you must define. On a grid, compare the beam's hit cell to the map. Give weight 3 if the map cell is occupied and the beam hit, weight 1 if both say free, and weight 0 if they disagree. You normalize weights 1, 3, and 6 by their sum, 10, before resampling. If every weight is 0, you reseed instead of dividing by 0.

Predict, weight, then resample.

1. Move all 100 particles

2. Weight each by the beam

3. Normalize by the sum

4. Draw 100 with replacement

Weights 1, 3, and 6

1. Three particles have unnormalized weights 1, 3, and 6. The sum is 10.

2. Normalized weights are 0.1, 0.3, and 0.6. In a draw of 100 you expect about 10, 30, and 60 copies.

3. A weight of 0 is never drawn. If every particle scores 0, you do not divide by the sum. You reseed.

Problem 1. Unnormalized weights are 1, 3, and 6. What are the normalized weights?

Answer. 0.1, 0.3, and 0.6. The sum is 10, and you divide each weight by 10.

Problem 2. Every particle has sensor weight 0. What do you do, and why not normalize?

Answer. Reseed. The sum is 0, so dividing would be undefined. The map and the motion model disagree completely.

Lecture 4 · 90 minutes

A path with a cost and a guess

You plan on the same grid. A free cell links to 4 neighbors, and a step into a free cell costs 1. If you allow unknown cells, charge them 2 so the path prefers mapped free space. Rank the frontier by f, which is g plus h: g is cost so far, h is remaining guess. On 4-connected cells, Manhattan distance never overestimates. The first time you pop the goal, the path is optimal for those costs.

An inadmissible h can make you expand the wrong cell and keep a longer path. If the true remaining cost is 3, an h of 5 is illegal, while an h of 3 is allowed. On a straight 4-connected hall, a cell 6 steps from the goal has Manhattan distance 6, not 4. You also block any cell with log odds above 0. A heuristic that walks through that occupied cell is scoring a graph you already forbade.

f is g plus a guess that must not lie high.

Cost so far

Free step costs 1

Unknown step costs 2

Occupied cells are blocked

Guess of the rest

Manhattan on 4 links

Never above the true cost

h of 5 exceeds cost 3

Expand the smaller f

1. Cell A has g of 4 and h of 3, so f is 7. Cell B has g of 5 and h of 1, so f is 6.

2. You expand B first. The path through B is not yet known to be shorter in g alone. The ranking uses f.

3. If h of A had been 5 while the true remainder was 3, that h is illegal for the optimality claim. Manhattan on a 6-step hall is 6, not 4.

Problem 1. Cell A has g 4 and h 3. Cell B has g 5 and h 1. Which do you expand first, and what are the two f values?

Answer. B first. f(A) is 7 and f(B) is 6.

Problem 2. The true remaining cost is 3 and your h is 5. Why can the first path you pop fail to be shortest?

Answer. h overestimates, so the optimality claim does not apply. An h of 3 would have been allowed. Manhattan distance on a 4-connected hall does not make this mistake.

Lecture 5 · 90 minutes

Layers that can suppress each other

Subsumption is a stack, not a plan. Give layer 1 the avoid behavior inside 0.4 meters, layer 2 a wander, and layer 3 a goal aim. Higher layers suppress lower ones for a cycle. A return at 0.3 meters lets layer 1 fire, and the aim toward a goal 2 meters away is dropped for that cycle. You get a quick flinch. You do not get the optimal grid path, and you should not claim that you did.

A potential field is a second fast law. Attraction of strength 2 pulls toward the goal, and repulsion grows inside 0.4 meters. You command the direction of the sum. Attraction (2, 0) plus repulsion (-3, 0) sums to (-1, 0), so you step away from the goal. That local trap is why a field is not a planner. If the heading flips for more than 5 cycles, hand the problem to the grid search instead of raising the gain.

A higher layer can silence a lower one.

Avoid inside 0.4 meters

Wander when nothing is close

Aim at a goal

Grid search after 5 flips

A flinch at 0.3 meters

1. The goal is 2 meters ahead. Layer 3 would aim at it. A sonar return is 0.3 meters, inside the 0.4 meter avoid band.

2. Layer 1 fires and suppresses the aim for this cycle. The command is the flinch, not the goal heading.

3. Attraction (2, 0) plus repulsion (-3, 0) is (-1, 0). You step away from the goal. If that heading flips for more than 5 cycles, you stop raising the gain and call the grid planner.

Problem 1. The goal is 2 meters ahead and a return is 0.3 meters. Which layer's command do you send this cycle?

Answer. Layer 1, the avoid layer. 0.3 is inside 0.4, so it suppresses the goal aim for this cycle. The result is not the optimal grid path.

Problem 2. Attraction is (2, 0) and repulsion is (-3, 0). Which way do you step, and why is that not a plan?

Answer. Toward (-1, 0), away from the goal. The sum is a local law. It can trap you, which is why a flip lasting more than 5 cycles should hand the problem to the grid search.

Lecture 6 · 90 minutes

A reward along a short path

A policy maps state to action and is trained on a reward you define, not on a layer diagram. Pay plus 10 for entering the goal and minus 1 for each action. The return of a trajectory is the sum. You keep the formula in one function so a change to the minus 1 is not buried in 3 call sites. You recompute a 4-action success as 6 before you trust a training curve.

The state you feed the policy has to be something the robot can measure. Integrated pose drifts, so do not train on a perfect x and y the robot will not have. Prefer a local observation: 8 range bins, the log-odds of the cell ahead, and the goal angle binned into 4 quadrants. That is 8 plus 1 plus 4 channels of input if you one-hot the quadrant, which is 13 numbers.

The return is a sum you can recompute.

1. Subtract 1 per action

2. Add 10 at the goal

3. Sum along the path

4. Train only on measured state

Four actions, return 6

1. Each action costs 1. Entering the goal pays 10. A trajectory with 4 actions that arrives on the last action returns 10 minus 4.

2. That return is 6. A trajectory with 12 actions returns 10 minus 12, which is minus 2.

3. The learner that maximizes the sum prefers 6 over minus 2. If you forgot a crash penalty, the formula you wrote is still the whole task, and the policy will take the crash.

Problem 1. The goal pays 10 and each action costs 1. What is the return of 4 actions that reach the goal?

Answer. 6. You compute 10 minus 4.

Problem 2. Why is a perfect x and y a bad training input if the robot only integrates wheel travel?

Answer. Wheel integration drifts, so the deployed robot will not see that perfect pose. Train on measurements it will have, such as 8 ranges, one log-odds cell, and 4 angle bins, 13 numbers in all.

Lecture 7 · 90 minutes

Beams that say which way is blocked

You turn sensors into the grid and the particle weights. In a toy lidar of 8 beams, a range under 0.5 meters blocks that direction. If 3 beams qualify, 3 directions are blocked and 5 are open. Do not steer into a blocked beam just because a cell last touched 20 cycles ago still says free. Write the new hit into the grid before the planner runs again. A mismatched beam must lower that particle's weight in the same cycle.

A camera is not a range sensor. If you place an object 2 meters ahead on a ground plane, that is 10 cells of 0.2 meters, and a height error of 0.5 meters can move the cell. Add plus 1 for a vision hit and plus 3 for a lidar hit, so one camera frame does not outvote 3 lidar misses. You record which sensor added each increment. Otherwise a bad ground plane quietly rewrites the map the planner trusts.

A fresh beam outranks a stale cell.

Read 8 ranges -> Block under 0.5 meters -> Write the grid -> Reweight particles

Three beams, then a camera

1. Eight beams return. Three are under 0.5 meters, so 3 directions are blocked and 5 are open.

2. You write those hits before planning. A cell last updated 20 cycles ago does not override a blocked beam.

3. A camera hit adds 1. Three lidar misses add minus 1 each, so minus 3. The net is minus 2. One camera frame does not outvote those 3 misses. A lidar hit would have added 3, which is a different event.

Problem 1. Of 8 beams, 3 are under 0.5 meters. How many directions are open?

Answer. 5. Eight minus 3 is 5.

Problem 2. A vision hit adds 1 and 3 lidar misses add minus 1 each. What is the net, and which sensor won?

Answer. The net is minus 2. The lidar misses win. One camera frame does not outvote 3 misses. You should also record which sensor wrote each increment.

Lecture 8 · 90 minutes

A robot that can explain a move

You can integrate a wheel step, update a 0.2 meter grid by plus 1 or minus 1, and resample 100 particles. You can search a 4-connected grid with Manhattan h, then let a beam inside 0.4 meters suppress that plan for one cycle. You can score 4 successful actions as 6 when the reward is plus 10 and minus 1. You should name which block emitted a bad command: odometry, map, particles, search, or policy.

A closed check mixes two of those blocks. Predict the pose with the wheel model, weight 3 particles by the latest beam, and only then plan. Do not plan from a particle whose cell has log odds above 0. If the beam inside 0.4 meters and the plan disagree, the reflex layer wins for 1 cycle and you replan. You are done when you can run that audit on a 10 meter map and name the failing block.

Name the block that emitted the command.

Wheel step to a pose

Grid plus particles

Plan or one-cycle flinch

Reward you can recompute

The heavy particle is occupied

1. Unnormalized weights 1, 3, and 6 normalize to 0.1, 0.3, and 0.6. The mode is the particle at 0.6.

2. That particle's cell has log odds above 0, so it is occupied. You do not start the 4-connected search there.

3. You resample or back up, then plan from a free cell. A beam inside 0.4 meters can still suppress the new plan for 1 cycle. The failure you name is the occupied start, not the heuristic.

Problem 1. A bad command sent the robot into a wall. What do you name before you retrain the policy?

Answer. The block that emitted it: odometry, the map, the particles, the search, or the policy. A 4-action return of 6 does not tell you which block was wrong.

Problem 2. Combine the particle weights with the planner. Weights 1, 3, and 6 normalize to 0.1, 0.3, and 0.6, and the 0.6 particle sits on log odds above 0. Do you start the search there?

Answer. No. The normalized weight 0.6 is the mode, but log odds above 0 means occupied. You resample or back up and plan from a free cell. The heuristic does not make an occupied cell legal.